



第3章 动态规划

DCampus

聂炎明
信息工程学院



■ 内容提纲:

- ❖ 3.1 概述
- ❖ 3.2 最长公共子序列问题
- ❖ 3.3 矩阵连乘问题
- ❖ 3.4 0-1背包问题
- ❖ 3.5 凸多边形最优三角剖分问题
- ❖ 3.6 最优二叉查找树问题等
- ❖ 补充 所有点对的最短路径问题



■ 要点：

- ❖ 动态规划法与分治法的异同；
- ❖ 动态规划法的基本要素与求解步骤；
- ❖ 动态规划法的应用。

■ 难点：

- ❖ 如何根据问题的最优子结构性性质构造构造动态规划方法中的递归公式或动态规划方程。



■ 示例：计算Fibonacci数：

$$F(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n \geq 2 \end{cases}$$

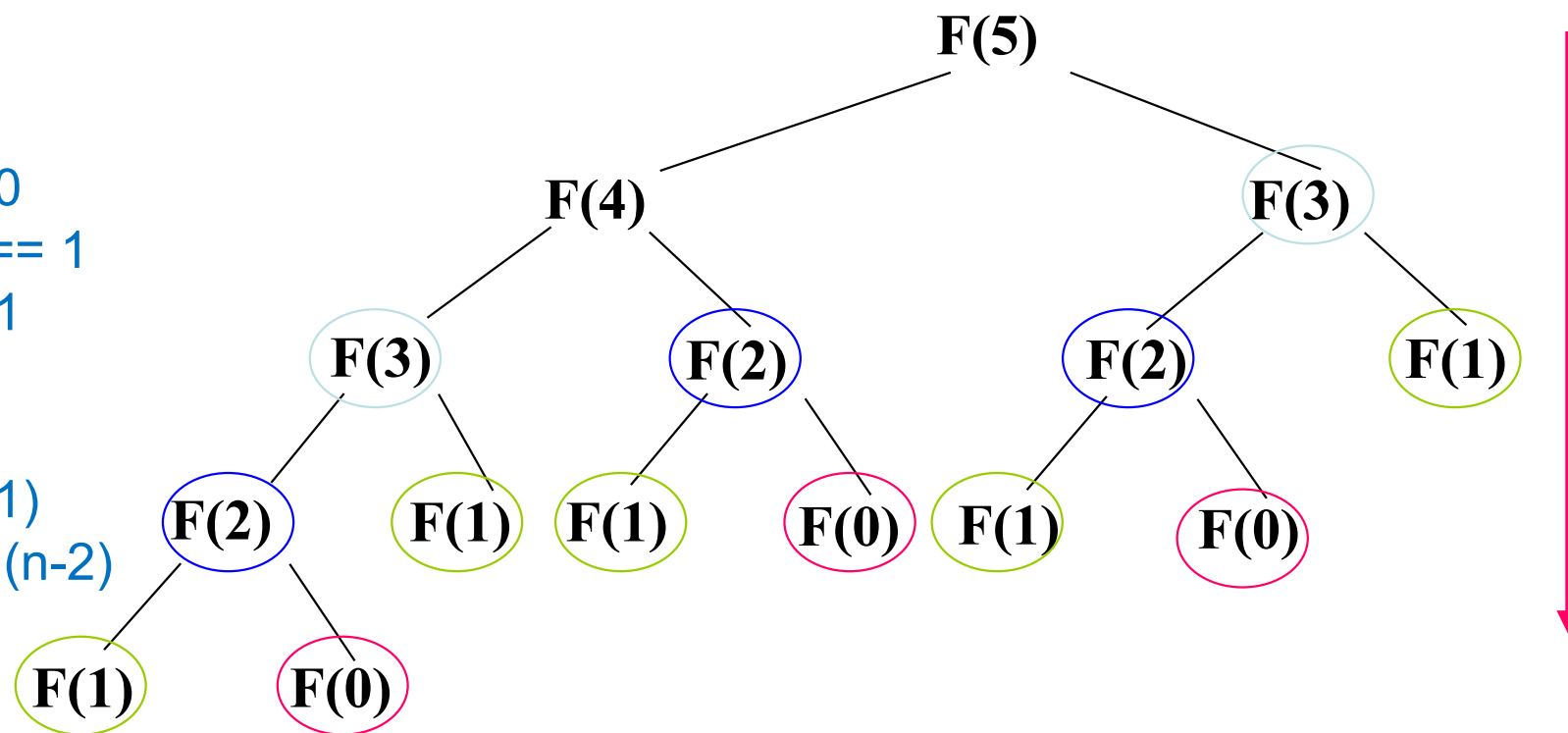
n	0	1	2	3	4	5	6	7	8	...
F(n)	0	1	1	2	3	5	8	13	21	...



■ 自顶向下递归求解:

伪代码:

```
FIB1(n)
IF n == 0
  RETURN 0
ELSE IF n == 1
  RETURN 1
ELSE
  RETURN
    FIB1(n-1)
    + FIB1(n-2)
```



- 许多子问题需要重复求解。当 n 很大时，效率很低。
- 时间复杂性为 $O(1.618^n)$ ，空间复杂性为 $O(n)$ 。



■ 自底向上并保存子问题值的求解:

伪代码:

FIB2(n)

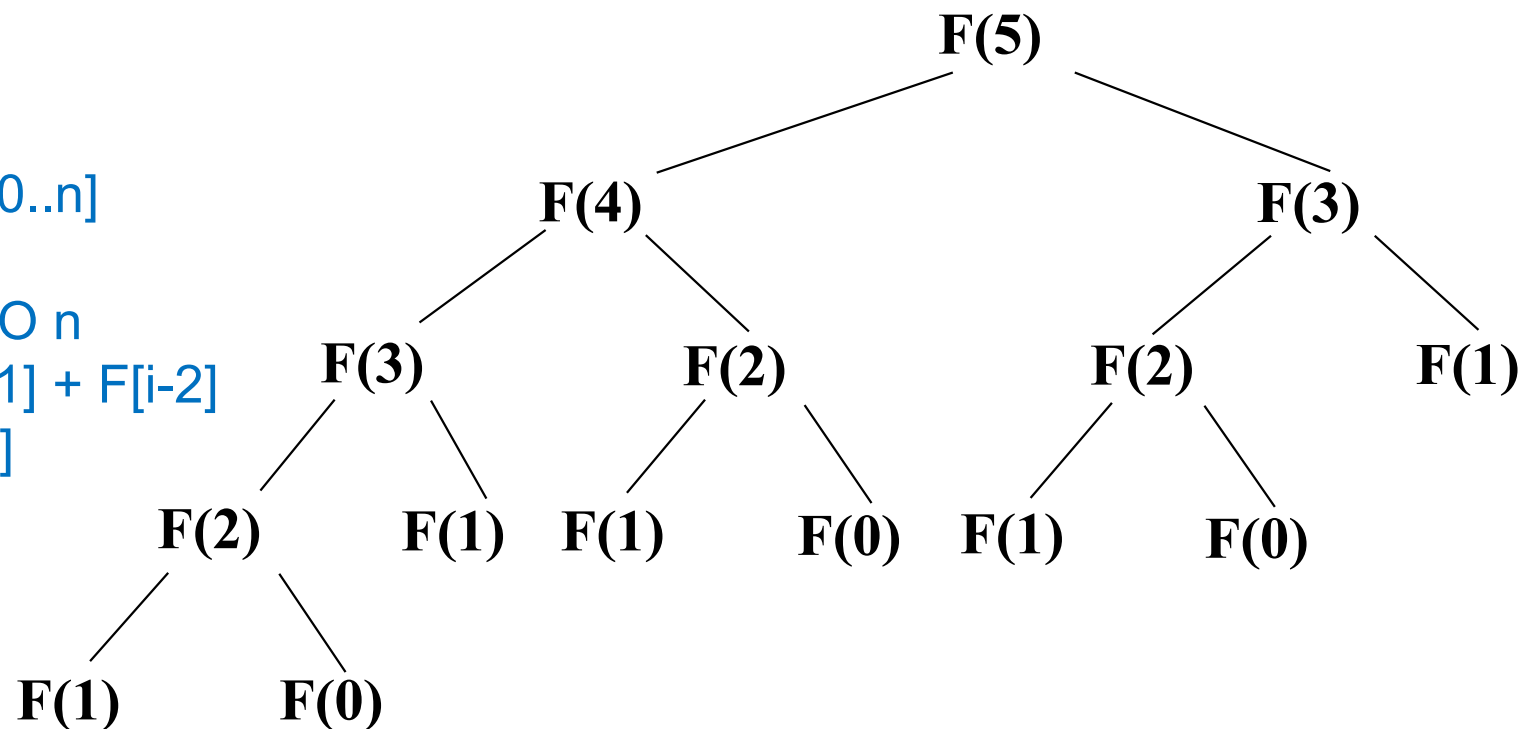
$F[0] \leftarrow 0$ // $F[0..n]$

$F[1] \leftarrow 1$

FOR $i \leftarrow 2$ TO n

$F[i] \leftarrow F[i-1] + F[i-2]$

RETURN $F[n]$



- 保存子问题的值，留作随后查找使用。
- 时间复杂性为 $O(n)$ ，空间复杂性为 $O(n)$ 。



- 动态规划（**Dynamic Programming, DP**）是一种多阶段决策优化方法，在数学、计算机科学和经济学中使用的。
- **20世纪50年代初**由美国数学家**R. E. Bellman**等提出。
- 动态规划背后的基本思想非常简单，类似于分治法。
- 但鉴于许多子问题通常可能是相同的，动态规划法试图对每个子问题仅求解一次，从而减少计算量。一旦某个给定子问题的解已经算出，则以填表方式将其存储，后续需要时可直接查表得到。



- **动态规划法的实质是分治思想和解决冗余：**
 - ❖ **分治思想：**将原问题分解为更小、更易求解的子问题，然后对子问题进行求解，并最终产生原问题的解。
 - ❖ **解决冗余：**求解过程中，所有子问题只求解一次并以表的方式保存，对于相同子问题并不重复求解而通过查表的方式获得。
- **动态规划法与分治法的异同：**
 - ❖ **相同点：**都基于分治思想；
 - ❖ **不同点：**分治法中各个子问题是独立的，而动态规划方法中允许子问题之间存在重叠。



■ 最优子结构性质：（ $F(5)$ 的最优解包括 $F(4)$ 的最优解）

- ❖ 问题最优解包含其子问题的最优解。动态规划的基础。基于最优子结构性性质导出递归公式或动态规划基本方程是解决一切动态规划问题的基本方法。反证法证明。

■ 子问题重叠性质：（ $F(3)$ 和 $F(2)$ 被多次求解）

- ❖ 求解过程中有些子问题出现多次而存在重叠。第一次遇到就加以解决并保存，若再次遇到时无需重复计算而直接查表得到，从而提高求解效率。该性质不是必要条件，但若无该性质该方法即没有优势。

■ 自底向上的求解方法：（Fibonacci数的第二种求解方法）

- ❖ 鉴于子问题重叠性质，采用自底向上的方法。先填写停止条件，求解每一级子问题并保存，直至得到原问题的解。



■ 动态规划法的求解步骤：

- ❖ 分析最优解的性质，以刻画最优解的结构特征。考察是否适合采用动态规划方法，即是否具备最优子结构性质；
- ❖ 递归定义最优值。即建立递归公式或动态规划方程，包括停止条件（递归出口）和递归体；
- ❖ 以自底向上的方式计算出最优值，并记录相关信息。应充分利用子问题重叠性质；
- ❖ 最终构造出最优解。



■ 备忘录方法为动态规划法的变形：

❖ 相同点：

- ✓ 采用表格保存已解决的子问题的值，在下次需要时直接查表而无需重新计算；

❖ 不同点：

- ✓ 备忘录方法采用自顶向下的递归方式，而动态规划法则采用自底向上的递归方式。
- ✓ 其控制结构与直接递归的控制结构相同，区别在于备忘录方法为每个已解的子问题建立备忘录以备需要时看，避免了相同子问题的重复求解。



■ 几个概念：

- ❖ **序列**：一个具有严格次序的对象元素的集合称为序列。字符对象的序列则为字符序列，如 $X = \{x_1, x_2, \dots, x_m\}$ 。
- ❖ **子序列**：给定序列 $X = \{x_1, x_2, \dots, x_m\}$ ，序列 $Z = \{z_1, z_2, \dots, z_k\}$ 是 X 的子序列，当且仅当存在一个严格递增的下标序列 $\{i_1, i_2, \dots, i_k\}$ ，对 $j \in \{1, 2, \dots, k\}$ 有 $z_j = x_{i_j}$ 。
- ❖ **前缀**：给定 $X = \{x_1, x_2, \dots, x_m\}$ ，子序列 $X_i = \{x_1, x_2, \dots, x_i\}$ 称为序列 X 的第 i 个前缀，其中 $i \in \{1, 2, \dots, m\}$ 。
- ❖ **公共子序列**：给定两个序列 X 和 Y ，当另一个序列 Z 既是 X 的子序列又是 Y 的子序列时，称 Z 是序列 X 和 Y 的公共子序列。
- ❖ **最长公共子序列 (Longest Common Subsequence, LCS) 问题**：给定两个序列 X 和 Y ，找出 X 和 Y 的一个最长公共子序列。



- 给定两个序列 $X = \{A, B, C, B, D, A, B\}$ 和 $Y = \{B, D, C, A, B, A\}$, 可以看出:
 - ❖ 序列 $\{B, C, A\}$ 是X和Y的一个公共子序列;
 - ❖ 序列 $\{B, C, B, A\}$ 是X和Y的一个公共子序列, 其长度是4, 而且它是X和Y的一个最长公共子序列;
 - ❖ $\{B, C, A, B\}$ 也是最长公共子序列。
 - ❖ 可见, 最长公共子序列并不唯一。而这里的LCS问题只要求出其中一个最长公共子序列即可。



■ 设序列 $X = \{x_1, x_2, \dots, x_m\}$ 和 $Y = \{y_1, y_2, \dots, y_n\}$ 的最长公共子序列为 $Z = \{z_1, z_2, \dots, z_k\}$ ，则：

- ❖ 若 $x_m = y_n$ ，则 $z_k = x_m = y_n$ ，且 Z_{k-1} 是 X_{m-1} 和 Y_{n-1} 的最长公共子序列；
- ❖ 若 $x_m \neq y_n$ 且 $z_k \neq x_m$ ，则 Z 是 X_{m-1} 和 Y 的最长公共子序列；
- ❖ 若 $x_m \neq y_n$ 且 $z_k \neq y_n$ ，则 Z 是 X 和 Y_{n-1} 的最长公共子序列。
- ❖ 以上，可以使用反证法进行证明。

■ 给定两个序列 $X = \{A, B, C, B, D, A, B\}$ 和 $Y = \{B, D, C, A, B, A\}$ ， $Z = \{B, C, B, A\}$ 是 X 和 Y 的一个最长公共子序列。

1



■ 建立LCS问题最优值的递归关系:

- ❖ 用 $c[i][j]$ 记录序列 X_i 和 Y_j 的LCS的长度。其中,
 $X_i = \{x_1, x_2, \dots, x_i\}$ 和 $Y_j = \{y_1, y_2, \dots, y_j\}$ 分别为序列 X 和 Y 的第 i 个前缀
和第 j 个前缀。
- ❖ 当 $i=0$ 或 $j=0$ 时, 空序列是 X_i 或 Y_j 的LCS, 此时 $c[i][j]=0$ 。
- ❖ 由最优子结构性质可建立递归关系。

$$c[i][j] = \begin{cases} 0 & i = 0 \parallel j = 0 \\ c[i-1][j-1] + 1 \text{ ①} & i, j > 0; x_i = y_j \\ \max \{c[i][j-1], c[i-1][j]\} & i, j > 0; x_i \neq y_j \end{cases}$$

② ←
③ ↑



■ LCS问题求解的具体设计如下：

- ❖ **步骤1：确定合适的数据结构。** 采用二维数组c来存放各个子问题的最优解，二维数组b记录各子问题最优值的来源（之前的递归公式上所附注）。
- ❖ **步骤2：初始化。** 令 $c[i][0]=0, c[0][j]=0$ ，其中， $0 \leq i \leq m, 0 \leq j \leq n$ 。
- ❖ **步骤3：循环阶段。** 根据递归关系式，确定 X_i 和 Y_j 的LCS的长度， $0 \leq i \leq m$ 。对于每个i循环， $0 \leq j \leq n$ 。
- ❖ **步骤4：构造解。** 根据二维数组b记录的信息，以自底向上的方式来构造该LCS问题的最优解。



■ 基于DP求解LCS问题的伪代码如下：

LCS-DP (X, Y)

FOR $i \leftarrow 0$ TO m // m 为 X 的长度

$c[i, 0] \leftarrow 0$

FOR $j \leftarrow 0$ TO n // n 为 Y 的长度

$c[0, j] \leftarrow 0$

FOR $i \leftarrow 1$ TO m

FOR $j \leftarrow 1$ TO n

IF $X[i] == Y[j]$

$c[i, j] \leftarrow c[i-1, j-1] + 1$

$b[i, j] \leftarrow 1$

ELSE IF $c[i-1, j] \geq c[i, j-1]$

$c[i, j] \leftarrow c[i-1, j]$

$b[i, j] \leftarrow 3$

ELSE

$c[i, j] \leftarrow c[i, j-1]$

$b[i, j] \leftarrow 2$

return c and b

❖ 该算法的时间复杂性： $O(mn) \rightarrow O(n^2)$ 。



■ 构造LCS问题最优解的伪代码如下：

```
LCS-PRINT (b, X, i, j)
```

```
    IF i == 0 OR j == 0
```

```
        RETURN
```

```
    IF b[i][j] == 1 
```

```
        LCS-PRINT (b, X, i-1, j-1)
```

```
        PRINT X[i]
```

```
    ELSE IF b[i][j] == 3 
```

```
        LCS-PRINT (b, X, i-1, j)
```

```
    ELSE
```

```
        LCS-PRINT (b, X, i, j-1) //b[i][j] == 2 
```

❖ 该算法的时间复杂性： $O(m+n) \rightarrow O(n)$ 。



■ LCS问题求解主过程的伪代码如下：

LCS-MAIN(X, Y) //X和Y为序列

$m \leftarrow \text{LENGTH}(X)$

$n \leftarrow \text{LENGTH}(Y)$

创建 $c[0..m, 0..n]$ //LCS的长度

创建 $b[0..m, 0..n]$ //可以不创建，直接基于c的求解过程构建

LCS-DP(X, Y, m, n)

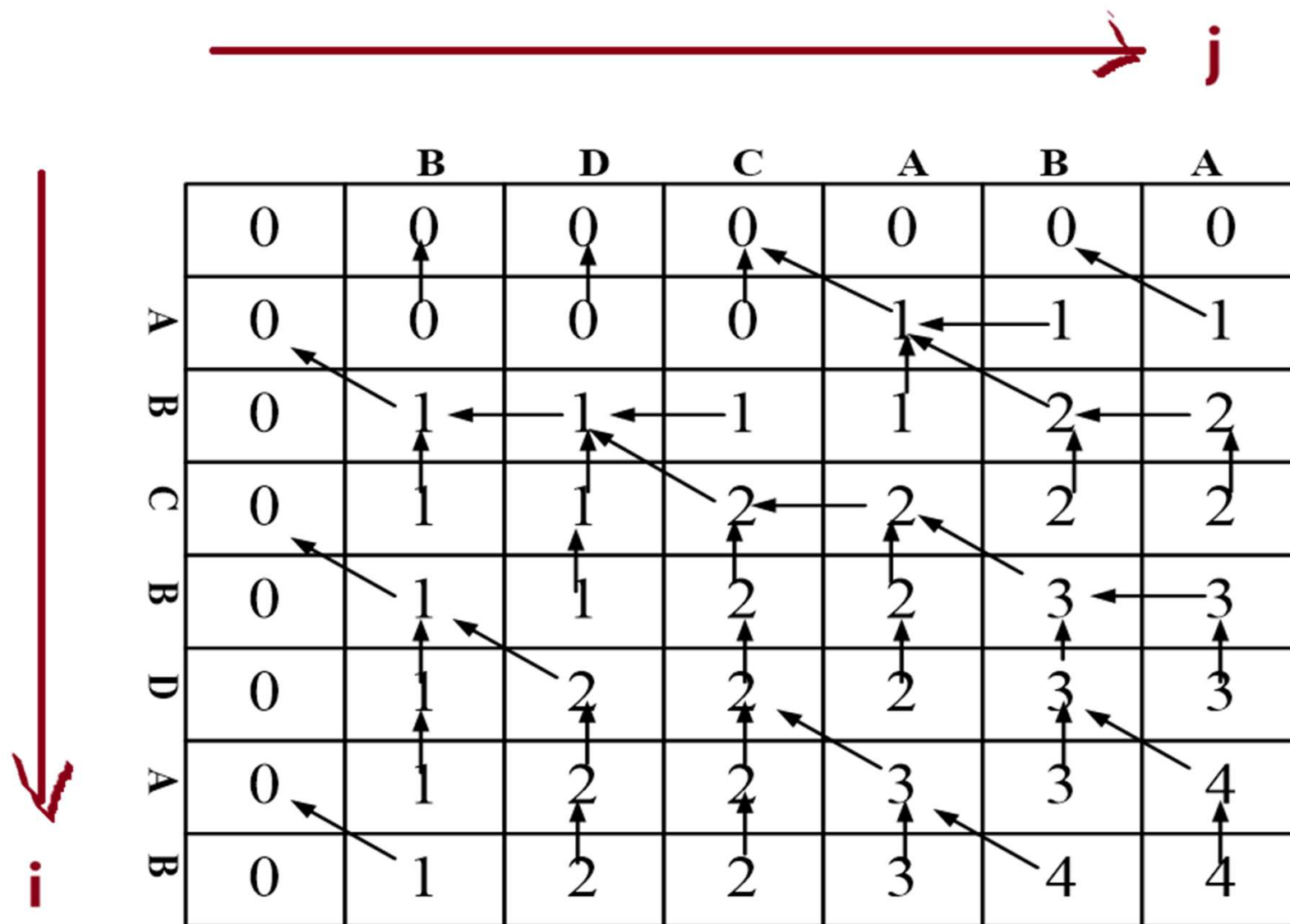
LCS-PRINT(b, X, m, n)

❖ 时间复杂性: $O(n^2) + O(n) \rightarrow O(n^2)$ 。

❖ 空间复杂性: $O(mn) \rightarrow O(n^2)$ 。



- 求序列 $X = \{A, B, C, B, D, A, B\}$ 和 $Y = \{B, D, C, A, B, A\}$ 的LCS:





■ 穷举搜索法：

- ❖ 枚举 $X_m = \{x_1, x_2, \dots, x_m\}$ 的所有子序列，并逐一检查每个子序列是否也为 $Y_n = \{y_1, y_2, \dots, y_n\}$ 的子序列，其中最长的即为 X_m 和 Y_n 的最长公共子序列。
- ❖ 该算法的时间复杂度为 $O(n2^m) \rightarrow O(n2^n)$ ，指数级。
- ❖ 该算法的空间复杂度为 $O(2^m) \rightarrow O(2^n)$ ，指数级。



■ 直接递归法求解LCS问题的伪代码如下：

```
LCS-REC(X, Y, i, j, c, b)
  IF i == 0 OR j == 0
    RETURN 0
  IF X[i] == Y[j]
    PRINT X[i]
    RETURN LCS-REC(X, Y, i-1, j-1, c, b) + 1
  ELSE
    RETURN MAX(LCS-REC(X, Y, i-1, j, c, b),
               LCS-REC(X, Y, i, j-1, c, b))
```

❖ 该算法的时间复杂性： $O(2^{\min(m, n)}) \rightarrow O(2^n)$ ，指数级。



■ 矩阵连乘的问题描述：

- ❖ 给定 n 个矩阵 $\{A_1, A_2, A_3, \dots, A_n\}$ 。其中， A_i 与 A_{i+1} ($i=1, 2, 3, \dots, n-1$) 是可乘的。如何确定计算矩阵连乘积的计算次序，使得依此次序计算矩阵连乘积所需要的数乘次数最少？
- ❖ 使用加括号的方法表示矩阵连乘的次序，不同加括号的方法所对应的计算次序是不同的。
 - ✓ 三个矩阵 $A_1A_2A_3$ 连乘共有两种加括号的方式： $(A_1A_2)A_3$ 和 $A_1(A_2A_3)$ ；
 - ✓ 四个矩阵 $A_1A_2A_3A_4$ 连乘共有五种加括号的方式： $(A_1(A_2(A_3A_4)))$ 、 $((A_1A_2)(A_3A_4))$ 、 $((A_1(A_2A_3))A_4)$ 、 $(A_1((A_2A_3)A_4))$ 及 $((A_1A_2)A_3)A_4$ 。



■ 矩阵连乘的递归定义：

❖ 完全加括号的矩阵连乘积可递归地定义为：

- ✓ 单个矩阵是完全加括号的；
- ✓ 矩阵连乘积A是完全加括号的，则A可以表示为2个完全加括号的矩阵连乘积B和C的乘积并加括号，形如 $A = (BC)$ 。

❖ 设现有四个矩阵 $A_{50 \times 10}$ ， $B_{10 \times 40}$ ， $C_{40 \times 30}$ 和 $D_{30 \times 5}$ ，共有五种完全加括号的方式：

- ✓ $(A((BC)D))$: $10 \times 40 \times 30 + 10 \times 30 \times 5 + 50 \times 10 \times 5 = 16000$
- ✓ $(A(B(CD)))$: $40 \times 30 \times 5 + 10 \times 40 \times 5 + 50 \times 10 \times 5 = \underline{10500}$
- ✓ $((AB)(CD))$: $50 \times 10 \times 40 + 40 \times 30 \times 5 + 50 \times 40 \times 5 = 36000$
- ✓ $((((AB)C)D))$: $50 \times 10 \times 40 + 50 \times 40 \times 30 + 50 \times 30 \times 5 = 87500$
- ✓ $((A(BC))D)$: $10 \times 40 \times 30 + 50 \times 10 \times 30 + 50 \times 30 \times 5 = 34500$



■ 采用穷举法求解矩阵连乘问题：

❖ 列举出所有可能的计算次序，并计算出每一种计算次序相应需要的数乘次数，从中找出一种数乘次数最少的计算次序。

❖ 设 n 个矩阵连乘的不同计算次序数为 $P(n)$ 。每种加括号的方式都可以分解为两个子矩阵的加括号问题：

$(A_1 \dots A_k) (A_{k+1} \dots A_n)$ ：

$$P(n) = \begin{cases} 1 & n=1 \\ \sum_{k=1}^{n-1} P(k)P(n-k) & n>1 \end{cases} \Rightarrow P(n) = \Omega(4^n / n^{3/2})$$

✓ $P(n)$ 为Catalan数，即 $P(n) = C(n-1) = \frac{1}{n} C_{2n-2}^{n-1}$ 。 **指数级**。不可行。



■ 采用穷举法求解矩阵连乘问题：

❖ 列举出所有可能的计算次序，并计算出每一种计算次序相应需要的数乘次数，从中找出一种数乘次数最少的计算次序。

❖ 设 n 个矩阵连乘的不同计算次序数为 $P(n)$ 。每种加括号的方式都可以分解为两个子矩阵的加括号问题：

$(A_1 \dots A_k) (A_{k+1} \dots A_n)$ ：

$$P(n) = \begin{cases} 1 & n=1 \\ \sum_{k=1}^{n-1} P(k)P(n-k) & n>1 \end{cases} \Rightarrow P(n) = \Omega(4^n / n^{3/2})$$

✓ $P(n)$ 为Catalan数，即 $P(n) = C(n-1) = \frac{1}{n} C_{2n-2}^{n-1}$ 。 指数级。



$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1s} \\ \vdots & \vdots & \ddots & \vdots \\ a_{i1} & a_{i2} & \cdots & a_{is} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{ms} \end{bmatrix} \cdot \begin{bmatrix} b_{11} & \cdots & b_{1j} & \cdots & b_{1n} \\ b_{21} & \cdots & b_{2j} & \cdots & b_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ b_{s1} & \cdots & b_{sj} & \cdots & b_{sn} \end{bmatrix}$$

$$= \begin{bmatrix} c_{11} & \cdots & a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{is}b_{sj} & \cdots & c_{1n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ c_{i1} & \cdots & c_{ij} & \cdots & c_{in} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ c_{m1} & \cdots & c_{mj} & \cdots & c_{mn} \end{bmatrix}$$

$A_{m \times s} \times B_{s \times n} = C_{m \times n}$



■ 矩阵连乘问题的最优子结构性质 - 反证法:

- ❖ 计算 $A[i:j]$ 的最优次序所包含的计算矩阵子链 $A[i:k]$ 和 $A[k+1:j]$ 的次序也是最优的 (证明如教材p52) :
 - ✓ 设计算 $A[i:j]$, $1 \leq i \leq j \leq n$, 所需要的最少数乘次数 $m[i, j]$, 则原问题的最优值为 $m[1, n]$;
 - ✓ 当 $i=j$, $A[i:j]=A[i:i]$ 。因此, $m[i, i]=0$, $i=1, 2, \dots, n$;
 - ✓ 当 $i < j$, $m[i, j] = m[i, k] + m[k+1, j] + p_{i-1}p_kp_j$, A_i 维数为 $p_{i-1} \times p_i$;
- ❖ 可以递归地定义 $m[i, j]$ 为:

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\} & i < j \end{cases}$$

- ✓ 其中, k 的位置只有 $j-i$ 种可能。

子问题重叠性质



递归求解矩阵连乘问题的伪代码：

RECURSIVE-MATRIX-CHAIN(p, i, j, m, s) // $m[n:n]$ 存最优值，其中的元素初始化为 ∞ ； $s[n:n]$ 记录加括号的位置 k 。这两个数组为输入/输出

IF $i == j$

$m[i, j] \leftarrow 0, s[i, j] \leftarrow 0$

RETURN 0

FOR $k = i$ TO $j-1$

$q \leftarrow \text{RECURSIVE-MATRIX-CHAIN}(p, i, k, m, s)$

+ $\text{RECURSIVE-MATRIX-CHAIN}(p, k+1, j, m, s)$

+ $p_{i-1}p_kp_j$

IF $q < m[i, j]$

$m[i, j] \leftarrow q$

$s[i, j] \leftarrow k$

RETURN $m[i, j]$ and $s[i, j]$

❖ 时间复杂性：

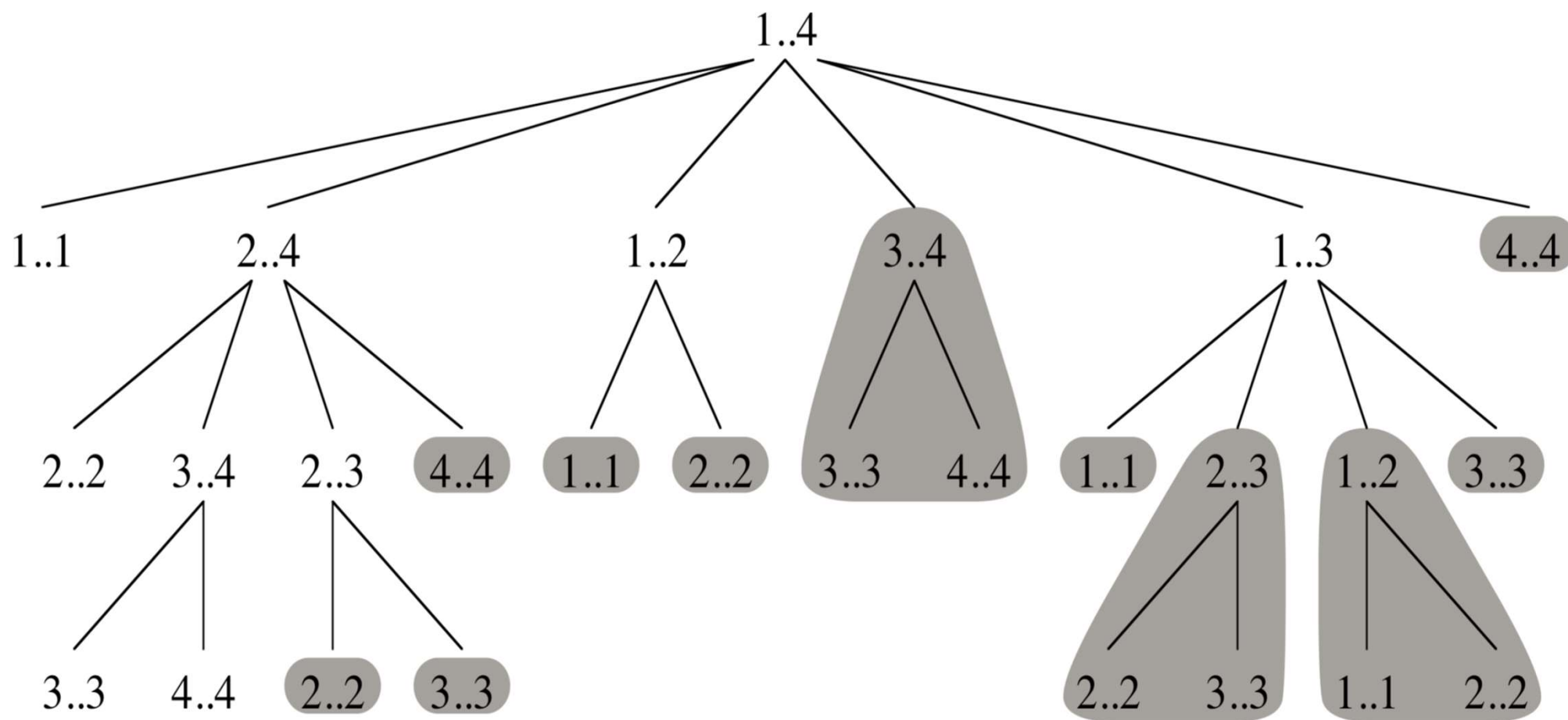
✓ $\Omega(4^n/n^{3/2})$

$A_1(p_0 \times p_1) * A_2(p_1 \times p_2) * \dots * A_i(p_{i-1} \times p_i) * A_{i+1}(p_i \times p_{i+1}) * \dots * A_{n-1}(p_{n-2} \times p_{n-1}) * A_n(p_{n-1} \times p_n)$

===》 $p[0..n]$



■ RECURSIVE-MATRIX-CHAIN(p , 1, 4)的递归调用树:





■ 采用DP求解矩阵连乘问题的伪代码：

MATRIX-CHAIN-ORDER-DP (n, p, m, s) // $p[1..n+1]$ 为 n 个矩阵的维数信息,
 $m[1..n, 1..n]$ 为最优值 (初值赋 ∞), $s[1..n, 1..n]$ 为最优决策

FOR $i \leftarrow 1$ TO n

$m[i, i] \leftarrow 0$

FOR $l \leftarrow 2$ TO n // l 为链的长度

 FOR $i \leftarrow 0$ TO $n-l+1$

$j \leftarrow i+l-1$

 FOR $k \leftarrow i$ TO $j-1$

$q \leftarrow m[i, k] + m[k+1, j] + p_{i-1}p_kp_j$

 IF $q < m[i, j]$

$m[i, j] \leftarrow q$

$s[i, j] \leftarrow k$

RETURN m AND s

❖ 时间复杂性:

✓ $O(n^3)$

❖ 空间复杂性:

✓ $O(1)$



■ 构造矩阵连乘问题最优解的伪代码：

```
PRINT-OPTIMAL-PARENS(s, i, j)
  IF i == j
    PRINT "A[" + i + "]"
  ELSE
    PRINT "("
    PRINT-OPTIMAL-PARENS(s, i, s[i, j])
    PRINT-OPTIMAL-PARENS(s, s[i, j]+1, j)
    PRINT ")"
```

❖ 时间复杂性：

✓ $O(n)$

❖ 空间复杂性：

✓ $O(n)$



■ 矩阵连乘问题求解主过程的伪代码：

MATRIX-CHAIN-ORDER-DP-MAIN(p, n) // p 为 n 个矩阵的维数信息

创建 $m[1..n, 1..n]$ // m 为最优值

创建 $s[1..n, 1..n]$ // s 为最优决策

$(m, s) \leftarrow \text{MATRIX-CHAIN-ORDER-DP}(n, p, m, s)$

PRINT-OPTIMAL-PARENS($s, 1, n$)

❖ 时间复杂性：

✓ $O(n^3) + O(n) \rightarrow O(n^3)$

❖ 空间复杂性：

✓ $O(n^2)$



- 计算矩阵连乘积 $A_1A_2A_3A_4A_5$ ，其维数如下：

A_1	A_2	A_3	A_4	A_5
3*2	2*5	5*10	10*2	2*3

❖ 计算矩阵连乘的最少次数：

- ✓ 设计算 $A[i:j]$ ， $1 \leq i \leq j \leq n$ ，所需要的最少数乘次数 $m[i,j]$ 。因此，原问题的最优值为 $m[1,n]$ 。
- ✓ 同时，记录对于问题 $A[i:j]$ 的最优（亦即数乘次数最少的加括号的位置序号 $s[i,j]$ ）。



3.3 矩阵连乘问题-DP求解示例

动态规划

A_1	A_2	A_3	A_4	A_5
3*2	2*5	5*10	10*2	2*3

(1) $m[i][i]=0$, $s[i][i]=0$;

(2) 当 $i=1$ 时,

$$m[1][2] = \min\{m[1][1] + m[2][2] + P_0P_1P_2 = 0 + 0 + 3 \times 2 \times 5 = 30\};$$

$$s[1][2]=1。$$

当 $i=2$ 时,

$$m[2][3] = \min\{m[2][2] + m[3][3] + P_1P_2P_3 = 0 + 0 + 2 \times 5 \times 10 = 100\};$$

$$s[2][3]=2。$$

依此类推, 求得 $m[3][4]=100, s[3][4]=3$, $m[4][5]=60$, $s[4][5]=4$ 。



3.3 矩阵连乘问题-DP求解示例 / 动态规划

(3) 当 $i=1$ 时,

A_1	A_2	A_3	A_4	A_5
3×2	2×5	5×10	10×2	2×3

$$m[1][3] = \min \begin{cases} m[1][1] + m[2][3] + P_0 P_1 P_3 = 0 + 100 + 3 \times 2 \times 10 = 160 \\ m[1][2] + m[3][3] + P_0 P_2 P_3 = 30 + 0 + 3 \times 5 \times 10 = 180 \end{cases} ;$$

$s[1][3]=1$ 。

当 $i=2$ 时,

$$m[2][4] = \min \begin{cases} m[2][2] + m[3][4] + P_1 P_2 P_4 = 0 + 100 + 2 \times 5 \times 2 = 120 \\ m[2][3] + m[4][4] + P_1 P_3 P_4 = 100 + 0 + 2 \times 10 \times 2 = 140 \end{cases} ;$$

$s[2][4]=2$ 。

当 $i=3$ 时,

$$m[3][5] = \min \begin{cases} m[3][3] + m[4][5] + P_2 P_3 P_5 = 0 + 60 + 5 \times 10 \times 3 = 210 \\ m[3][4] + m[5][5] + P_2 P_4 P_5 = 100 + 0 + 5 \times 2 \times 3 = 130 \end{cases} ;$$

$s[3][5]=4$ 。



3.3 矩阵连乘问题-DP求解示例

动态规划

A_1	A_2	A_3	A_4	A_5
3×2	2×5	5×10	10×2	2×3

(4) 当 $i=1$ 时,

$$m[1][4] = \min \begin{cases} m[1][1] + m[2][4] + P_0 P_1 P_4 = 0 + 120 + 3 \times 2 \times 2 = 132 \\ m[1][2] + m[3][4] + P_0 P_2 P_4 = 30 + 100 + 3 \times 5 \times 2 = 160 \\ m[1][3] + m[4][4] + P_0 P_3 P_4 = 160 + 0 + 3 \times 10 \times 2 = 220 \end{cases} ;$$

$s[1][4]=1$ 。

当 $i=2$ 时,

$$m[2][5] = \min \begin{cases} m[2][2] + m[3][5] + P_1 P_2 P_5 = 0 + 130 + 2 \times 5 \times 3 = 160 \\ m[2][3] + m[4][5] + P_1 P_3 P_5 = 100 + 60 + 2 \times 10 \times 3 = 220 \\ m[2][4] + m[5][5] + P_1 P_4 P_5 = 120 + 0 + 2 \times 2 \times 3 = 132 \end{cases} ;$$

$s[2][5]=4$ 。



3.3 矩阵连乘问题-DP求解示例

动态规划

A_1	A_2	A_3	A_4	A_5
3*2	2*5	5*10	10*2	2*3

$$(5) \quad m[1][5] = \min \begin{cases} m[1][1] + m[2][5] + P_0 P_1 P_5 = 0 + 132 + 3 \times 2 \times 3 = 150 \\ m[1][2] + m[3][5] + P_0 P_2 P_5 = 30 + 130 + 3 \times 5 \times 3 = 205 \\ m[1][3] + m[4][5] + P_0 P_3 P_5 = 160 + 60 + 3 \times 10 \times 3 = 310 \\ m[1][4] + m[5][5] + P_0 P_4 P_5 = 132 + 0 + 3 \times 2 \times 3 = 150 \end{cases}$$

$s[1][5] = 1$ 或

$s[1][5] = 4$



3.3 矩阵连乘问题-DP求解示例

动态规划

■ 最优值 $m[i][j]$:

$m[i][j]$

	A_1	A_2	A_3	A_4	A_5
A_1	0	30	160	132	150
A_2		0	100	120	132
A_3			0	100	130
A_4				0	60
A_5					0

$s[i][j]$

	A_1	A_2	A_3	A_4	A_5
A_1	0	1	1	1	1
A_2		0	2	2	4
A_3			0	3	4
A_4				0	4
A_5					0

■ 最优决策 $s[i][j]$:

■ 解为:

❖ $A_1((A_2(A_3A_4))A_5)$



■ 基于备忘录方法求解矩阵连乘问题的伪代码：

LOOK-CHAIN-MEMO(m, s, p, i, j) // $m[n:n]$ 存最优值，其中的元素初始化为 ∞ ； $s[n:n]$ 记录加括号的位置 k

IF $m[i, j] > 0$ return $m[i, j]$

IF $i == j$ return 0

$u \leftarrow$ LOOK-CHAIN-MEMO(m, s, p, i, i)

+ LOOK-CHAIN-MEMO($m, s, p, i+1, j$) + $p_{i-1}p_ip_j$ // i 处加括号

$s[i, j] \leftarrow i$

FOR $k \leftarrow i+1$ TO $j-1$

$q \leftarrow$ LOOK-CHAIN-MEMO(m, s, p, i, k)

+ LOOK-CHAIN-MEMO($m, s, p, k+1, j$) + $p_{i-1}p_kp_j$ // k 处加括号

IF $q < u$

$u \leftarrow q$

$s[i, j] \leftarrow k$ // 最优决策

$m[i, j] \leftarrow u$

RETURN u

❖ 时间复杂性：

✓ $O(n^3)$

❖ 空间复杂性：

✓ $O(1)$



■ 基于备忘录方法求解矩阵连乘问题主过程的伪代码：

MATRIX-CHAIN-MEMO-MAIN(p, n) // p 为 n 个矩阵的维数信息

创建 $m[1..n, 1..n]$ // m 为最优值

创建 $s[1..n, 1..n]$ // s 为最优决策

FOR $i \leftarrow 1$ TO n

 FOR $j \leftarrow 1$ TO n

$m[i, j] \leftarrow 0$

$(m, s) \leftarrow \text{LOOK-CHAIN-MEMO}(m, s, p, 1, n)$

PRINT-OPTIMAL-PARENS($s, 1, n$)

❖ 时间复杂性：

✓ $O(n^3)$

❖ 空间复杂性：

✓ $O(n^2)$



■ 问题描述:

❖ n 个物品和1个背包。对物品 i ，其价值为 v_i ，重量为 w_i ，背包容量为 W 。如何选取物品装入背包，使背包中所装入的物品的总价值最大？这里， w_i ， W 都是正整数。

❖ 约束条件:

$$\begin{cases} \sum_{i=1}^n w_i x_i \leq W \\ x_i \in \{0,1\}, (1 \leq i \leq n) \end{cases}$$

❖ 目标函数:

$$\max \sum_{i=1}^n v_i x_i$$



■ 0-1背包问题的最优子结构性质 - 反证法:

❖ 设 $(x_1, x_2, \dots, x_n)$ 是所给0-1背包问题的一最优解, 则 $(x_2, \dots, x_n)$ 是下面相应子问题的一个最优解 (证明可参考王秋芬老师的教材p75-76):

✓ 约束条件:

$$\begin{cases} \sum_{i=2}^n w_i x_i \leq W - w_1 x_1 \\ x_i \in \{0, 1\}, (2 \leq i \leq n) \end{cases}$$

✓ 目标函数:

$$\max \sum_{i=2}^n v_i x_i$$

✓ 注意: $(x_1, \dots, x_{n-1})$ 子问题, 同上。



■ 0-1背包问题的最优值递推关系：

- ❖ 从规模最大的问题的最优解与其子问题的最优解的关系，一级一级抽象问题的一般表述，得出递推关系式：

$$C[0][j]=C[i][0]=0$$

$$C[i][j]=\begin{cases} C[i-1][j] & j < w_i \\ \max\{C[i-1][j], C[i-1][j-w_i]+v_i\} & j \geq w_i \end{cases}$$

✓ $C[i][j] = \max \sum_{k=1}^i v_k x_k$, 为 $\begin{cases} \sum_{k=1}^i w_k x_k \leq j \\ x_k \in \{0,1\} \quad 1 \leq k \leq i \end{cases}$ 的最优值。



■ 采用DP求解0-1背包问题的伪代码：

KNAPSACK-01-DP (w , v , W) // w 为重量, v 为价值, W 为容量

// $C[0..n, 0..W]$ 为最优值

FOR $i \leftarrow 1$ TO n

$C[i, 0] \leftarrow 0$

FOR $j \leftarrow 1$ TO W

$C[0, j] \leftarrow 0$

FOR $i \leftarrow 1$ TO n

 FOR $j \leftarrow 1$ TO W

 IF $j < w[i]$

$C[i, j] \leftarrow C[i-1][j]$

 ELSE

$C[i, j] \leftarrow \text{MAX}\{C[i-1][j], C[i-1][j-w[i]] + v[i]\}$

RETURN C

❖ 时间复杂性:

✓ $O(nW)$

❖ 空间复杂性:

✓ $O(1)$



■ 0-1背包问题的最优解构建的伪代码：

KNAPSACK-TRACEBACK-01 (w, W, C, X) // w 为重量, W 为容量, C 为最优值

// $X[1..n]$ 为构建的最优解

$n \leftarrow w.length - 1$

$j \leftarrow W$ // Remained Capacity

FOR $i \leftarrow n$ TO 1

 IF $C[i][j] == C[i-1][j]$

$X[i] \leftarrow 0$

 ELSE

$X[i] \leftarrow 1$

$j \leftarrow j - w[i]$

RETURN X

❖ 时间复杂性:

✓ $O(n)$

❖ 空间复杂性:

✓ $O(1)$



■ 0-1背包问题求解主过程的伪代码：

KNAPSACK-01-DP-MAIN (w, v, W) // w 为重量, v 为价值, W 为容量

创建 $C[1..n, 1..W]$ // m 为最优值

KNAPSACK-01-DP (w, v, W, C)

KNAPSACK-TRACEBACK (w, v, W, C, X) // $X[1..n]$ 为构建的最优解

❖ 时间复杂性:

✓ $O(nW) + O(n) \rightarrow O(nW)$

❖ 空间复杂性:

✓ $O(nW) + O(1) \rightarrow O(nW)$



■ 示例:

❖ $w = (2, 2, 6, 5, 4)$, $v = (6, 3, 5, 4, 6)$, $W = 10$ 。

$i \backslash j$	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	6	6	6	6	6	6	6	6	6
2	0	0	6	6	9	9	9	9	9	9	9
3	0	0	6	6	9	9	9	9	11	11	14
4	0	0	6	6	9	9	9	10	11	13	14
5	0	0	6	6	9	9	12	12	15	15	15

❖ 最优解:

✓ $X = (x_1, x_2, x_3, x_4, x_5) = (1, 1, 0, 0, 1)$



■ 算法存在的问题：

- ❖ 假设物品的重量 w_i ($1 \leq i \leq n$) 为整数：
 - ✓ 如果 w_i ($1 \leq i \leq n$) 为连续型的实数，如何枚举？
- ❖ 该算法复杂性目前为 $O(nW)$ ：
 - ✓ 当背包容量 W 很大时，算法所需的计算时间较多。
 - ✓ 若 $W > 2^n$ ，算法复杂性则为 $O(n2^n)$ 。指数级

■ 算法改进的机会：

- ❖ 对于每个给定 i ($1 \leq i \leq n$)，存放最优值的 $C[i][j]$ 对于 j 为**阶梯状单调非减函数**（当 j 为连续变量时，亦成立）。
- ❖ 因而，只需要考虑部分特殊点（亦即**跳跃点**）。



■ 算法的改进（参考王秋芬p106-107）：

❖ 阶梯状单调非减函数 $C[i][j]$ 中值发生跳跃改变的点被称为跳跃点，记为 $p[i] \rightarrow (j, C[i][j])$ 。

❖ $p[i]$ 按 j 非减序排列，可知 $C[i][j]$ 亦是非减序的。

❖ $C[i][j] \leftarrow \max \{C[i-1][j], C[i-1][j-w_i]+v_i\}$ ，式右边两项分别对应于 $p[i-1]$ 和 $q[i-1]$ 。

由此，可计算出 $p[i]$ ：

✓ $p[i] \leftarrow p[i-1] \cup q[i-1]$

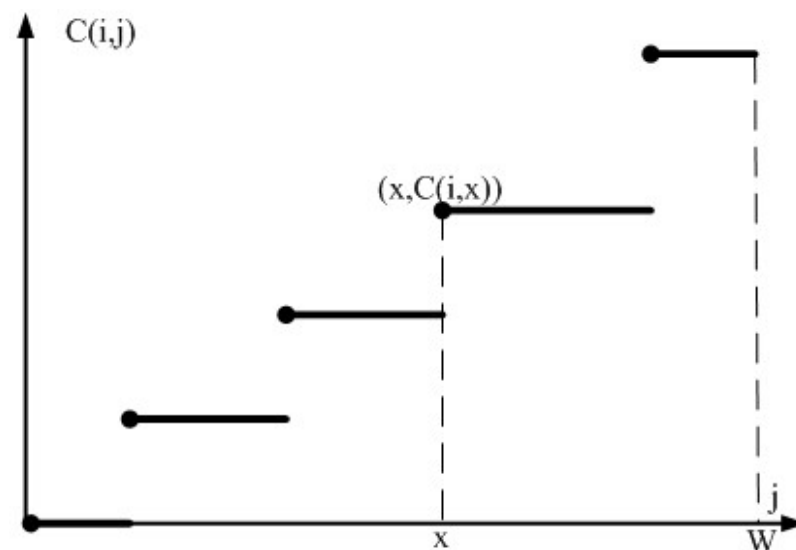
$q[i-1]$ 可由 $p[i-1]$ 计算出：

✓ $q[i-1] \leftarrow p[i-1] \oplus (w_i, v_i)$

初始时：

✓ $p[0] \leftarrow \{(0, 0)\}$

❖ 受控跳跃点 与 无效跳跃点。





■ 采用DP求解0-1背包问题改进方法伪代码：

KNAPSACK-01-OPT (w, v, W) // w 为重量, v 为价值, W 为容量

// $p[1..n, 2]$: 存放指定 i 的跳跃点即 $(j, C[i][j])$ 值对。 $q[1..n, 2]$: 如定义

$p[0] \leftarrow \{(0, 0)\}$

FOR $i \leftarrow 1$ to n

$q[i-1] \leftarrow p[i-1] \oplus (w[i], v[i])$

$p[i] \leftarrow p[i-1] \cup q[i-1]$

PROCESS ($p[i], W$) // 剔除受控/无效的跳跃点

RETURN p



❖ 时间复杂性:

✓ $O(\min(nW, 2^n))$

❖ 空间复杂性:

✓ $O(W)$



■ 采用DP求解0-1背包问题改进方法之构建解的伪代码：

BACKTRACK-OPT (w, v, p, n) // w 为重量, v 为价值, p 为跳跃点集, $x[1..n]$ 存放解

$j \leftarrow \text{MAX}(p[n][0]); m \leftarrow \text{MAX}(p[n][1])$

FOR $i \leftarrow n$ TO 1

//遍历 $p[i-1]$, 如其中之一满足

IF $(j, m) == p[i-1] \oplus (w[i], v[i])$

$x[i] \leftarrow 1; j \leftarrow p[i-1][0]; m \leftarrow p[i-1][1]$

ELSE

$x[i] \leftarrow 0$

RETURN x

❖ 时间复杂性:

✓ $O(n)$

❖ 空间复杂性:

✓ $O(1)$



$w = (2, 2, 6, 5, 4)$, $v = (6, 3, 5, 4, 6)$, $W = 10$ 。

■ 求解过程（剔除 受控跳跃点 和 无效跳跃点）：

- ❖ $p[0] \leftarrow \{(0,0)\}$;
- $q[0] \leftarrow p[0] \oplus (w_1, v_1) = \{(0,0)\} \oplus (2,6) = \{(2,6)\}$;
- ❖ $p[1] \leftarrow p[0] \cup q[0] = \{(0,0), (2,6)\}$;
- $q[1] \leftarrow p[1] \oplus (w_2, v_2) = \{(2,3), (4,9)\}$;
- ❖ $p[2] \leftarrow p[1] \cup q[1] = \{(0,0), \underline{(2,3)}, (2,6), (4,9)\}$;
- $q[2] \leftarrow p[2] \oplus (w_3, v_3) = \{(6,5), (8,11), (10,14)\}$;
- ❖ $p[3] \leftarrow p[2] \cup q[2] = \{(0,0), (2,6), (4,9), \underline{(6,5)}, (8,11), (10,14)\}$;
- $q[3] \leftarrow p[3] \oplus (w_4, v_4) = \{(5,4), (7,10), (9,13), \underline{(13,15)}, \underline{(15,18)}\}$;
- ❖ $p[4] \leftarrow p[3] \cup q[3] = \{(0,0), (2,6), (4,9), \underline{(5,4)}, (7,10), (8,11), (9,13), (10,14)\}$;
- $q[4] \leftarrow p[4] \oplus (w_5, v_5) = \{(4,6), (6,12), (8,15), \underline{(11,16)}, \underline{(12,17)}, \underline{(13,19)}, \underline{(14,20)}\}$;
- ❖ $p[5] \leftarrow p[4] \cup q[4] = \{(0,0), (2,6), (4,9), \underline{(4,6)}, (6,12), \underline{(7,10)}, \underline{(8,11)}, (8,15), \underline{(9,13)}, \underline{(10,14)}\}$;
- ❖ 最终, $p[5] \leftarrow \{(0,0), (2,6), (4,9), (6,12), (8,15)\}$ 。



$w = (2, 2, 6, 5, 4)$, $v = (6, 3, 5, 4, 6)$, $W = 10$ 。

■ 构造解的过程:

- ❖ $p[5] = \{ (0,0), (2,6), (4,9), (6,12), (8,15) \}$
- ❖ $p[4] = \{(0,0), (2,6), (4,9), (7,10), (8,11), (9,13), (10,14)\}$ 。由于 $p[4]$ 中的 $(4,9) \oplus (4,6) = (8,15)$ (最优值), 故 $x_5 \leftarrow 1$, $j \leftarrow 4$, $m \leftarrow 9$;
- ❖ $p[3] = \{(0,0), (2,6), (4,9), (8,11), (10,14)\}$ 。由于 $p[3]$ 中的所有点 $\oplus$ $(w_4, v_4) \neq (j, m)$, 故 $x_4 \leftarrow 0$;
- ❖ $p[2] = \{(0,0), (2,6), (4,9)\}$ 。 $p[2]$ 中的所有点 $\oplus$ $(w_3, v_3) \neq (j, m)$, 故 $x_3 \leftarrow 0$;
- ❖ $p[1] = \{(0,0), (2,6)\}$ 。由于 $p[1]$ 中的 $(2,6) \oplus (2,3) = (4,9)$, 所以 $x_2 \leftarrow 1$, $j \leftarrow 2$, $m \leftarrow 6$;
- ❖ $p[0] = \{(0,0)\}$ 。由于 $p[0]$ 中的 $(0,0) \oplus (2,6) = (2,6)$, 故 $x_1 \leftarrow 1$, $j \leftarrow 0$, $m \leftarrow 0$ 。
- ❖ 最优解: $X(x_1, x_2, x_3, x_4, x_5) \leftarrow (1, 1, 0, 0, 1)$



■ 问题描述:

❖ n 个物品和1个背包。对物品 i ，其价值为 v_i ，重量为 w_i ，背包容量为 W 。如何选取（部分）物品装入背包，使背包中所装入的物品的总价值最大？其中， w_i ， W 都是正整数。

❖ 约束条件：
$$\begin{cases} \sum_{i=1}^n w_i x_i \leq W \\ x_i \in [0, 1.0], (1 \leq i \leq n) \end{cases}$$

.....

❖ 目标函数：
$$\max \sum_{i=1}^n v_i x_i$$



■ 背包问题的最优子结构性质 - 反证法:

❖ 设 $(x_1, x_2, \dots, x_n)$ 是所给0-1背包问题的一最优解, 则 $(x_2, \dots, x_n)$ 是下面相应子问题的一个最优解:

✓ 约束条件:

$$\begin{cases} \sum_{i=1}^n w_i x_i \leq W \\ x_i \in [0, 1.0], (1 \leq i \leq n) \end{cases}$$

✓ 目标函数: $\max \sum_{i=2}^n v_i x_i$



■ 背包问题的贪心选择性质 – 归纳法：

- ❖ 在不超过背包的容量的情况下，尽可能多（指重量，如全部或部分）地选择更为贵重（指单位重量的价值最大）的物品，直至装满背包为止。
- ❖ 参考“第4章 贪心法”。



■ 基于贪心法求解背包问题的伪代码如下：

```
GREEDY-KNAPSACK (w, v, W) //w为重量, v为价值, W为容量
    SORT-ASC-BY-V (v, w) //按照物品的单位价值非降序排序, 同时调整v和w
    n ← w.length
    //x[1..n]为最优值, 存放已选物品的相应重量。各元素初始化为0
    wRemained ← W
    FOR i ← 1 TO n
        IF wRemained ≤ 0
            BREAK;
        ELSE IF wRemained ≥ w[i]
            x[i] ← 1
            wRemained ← wRemained - w[i]
        ELSE
            x[i] ← 0
            BREAK
    RETURN x
```

❖ 时间复杂性:

✓ $O(n \log n)$

❖ 空间复杂性:

✓ $O(1)$



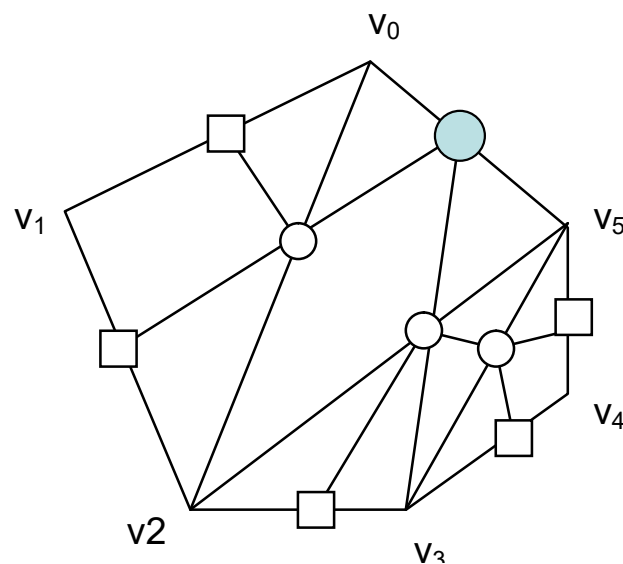
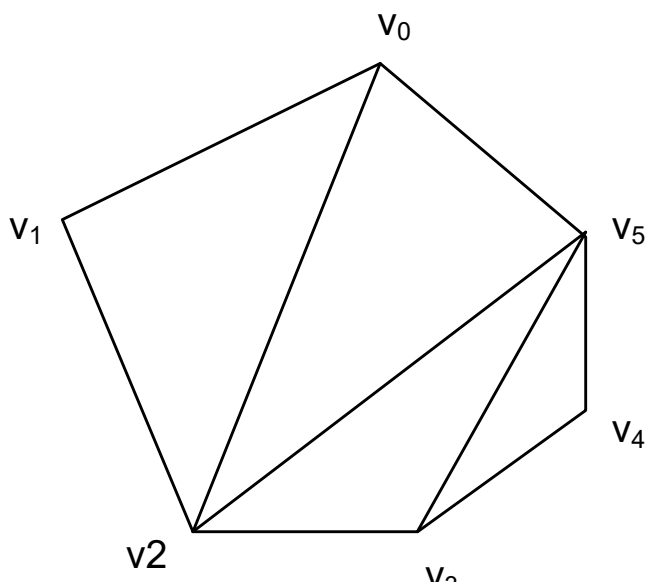
■ 凸多边形最优三角剖分的几个概念：

- ❖ 凸多边形：一个简单多边形及其内部构成一个闭凸集时，称该简单多边形为凸多边形，即凸多边形边界上或内部的任意两点所连成的直线段上所有点均在凸多边形的内部或边界上。
- ❖ 凸多边形的弦：凸多边形的不相邻的两个顶点连接的直线段称为凸多边形的弦。可为弦指定带权函数。
- ❖ 凸多边形的三角剖分 (Convex Polygon Triangulation, CPT)：是指将一个凸多边形分割成互不相交的三角形的弦的集合。



■ 凸多边形最优三角剖分的几个概念：

- ❖ 凸多边形的三角剖分：凸多边形的三角剖分指将一个凸多边形分割成互不相交的三角形的弦的集合。
- ❖ 凸六边形的一种三角剖分及其对应的二叉树结点，如：





■ 凸多边形最优三角剖分的问题描述：

❖ 凸多边形的最优三角剖分 (Optimal Convex Polygon Triangulation, OCPT): 给定凸多边形及定义在边、弦构成的三角形的权函数，最优三角剖分是指不同剖分方法所划分的各三角形上权函数之和最小的三角剖分。

❖ 可以定义权函数为：

✓ $w(v_i, v_j, v_k) = |v_i v_j| + |v_j v_k| + |v_k v_i|$



OCPT的最优子结构性质分析:

- ❖ 设 $v_0v_kv_n$ 是将 $n+1$ 边形 $P=\{v_0, v_1, \dots, v_n\}$ 分成三部分 $\{v_0, v_1, \dots, v_k\}$ 、 $\{v_k, v_{k+1}, \dots, v_n\}$ 和 $\{v_0, v_k, v_n\}$ 的最佳剖分方法, 那么凸多边形 $\{v_0, v_1, \dots, v_k\}$ 的剖分一定是最优的, $\{v_k, v_{k+1}, \dots, v_n\}$ 的剖分也一定是最优的。
- ❖ 可以采用反证法证明 (参考教材p63)。

OCPT的最优值递归关系:

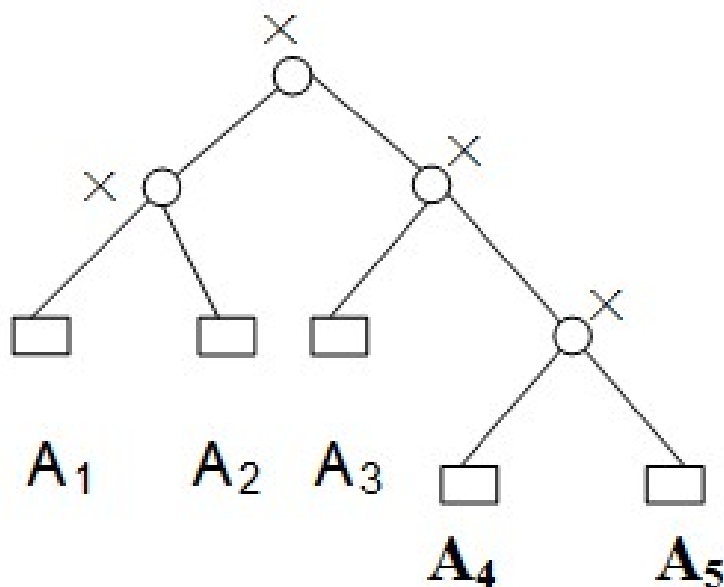
- ❖ 设 $m[i][j]$ 表示 $v_{i-1}v_i\dots v_j$ 最优三角剖分权函数之和。当 $i=j$ 时, 表示一条直线段, 将其看作退化多边形, 而且其权函数为0。则:

$$m[i][j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i][k] + m[k+1][j] + w(v_{i-1}v_kv_j)\} & i < j \end{cases}$$

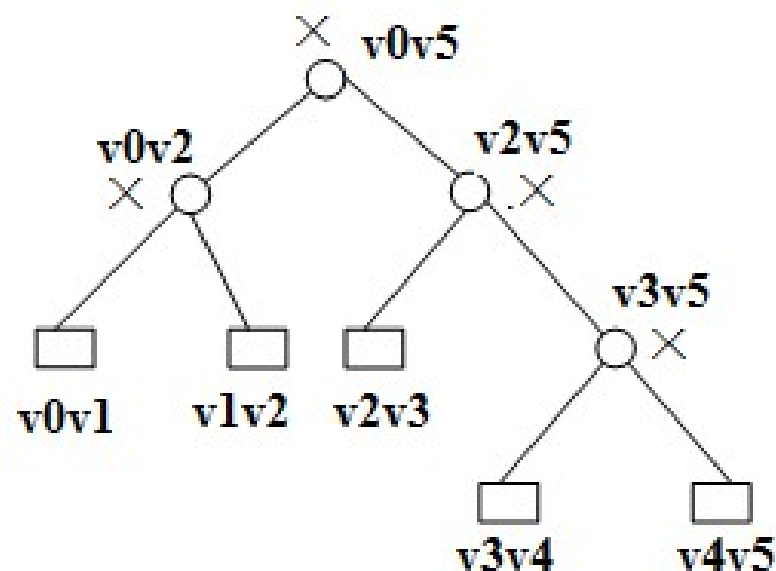


■ CPT问题与矩阵连乘问题：

- ❖ 如同矩阵连乘的计算次序等价于矩阵连乘的完全加括号方式一样，CPT与矩阵连乘表达式加括号方式之间具有十分紧密的联系。皆可对应于一棵二叉树。



矩阵连乘 $((A_1 A_2)(A_3(A_4 A_5)))$



凸多边形三角剖分



■ 二叉查找树:

- ❖ 给定 n 个关键字组成的有序序列 $S = \{s_1, s_2, \dots, s_n\}$ ($s_1 < s_2 < \dots < s_n$), 用这些关键字建立一棵**二叉查找树** (Binary Search Tree, BST) T 。 T 中的 s_i ($1 \leq i \leq n$) 为**实结点** (内部结点), 对应关键字 s_i , 其相应的查找概率为 p_i 。
- ❖ S 中可能不存在某些待查值, 因此在 T 中设置 $n+1$ 个**虚结点** e_i ($0 \leq i \leq n$) 为不在 S 中的伪关键字。其中, e_0 表示小于 s_1 的所有值, e_n 表示大于 s_n 的所有值, e_i 表示位于 s_i 与 s_{i+1} 之间的所有值。每个虚结点 e_i 对应一个查找概率 q_i 。
- ❖ T 中的每次查找要么成功 (即检索到实结点 s_i), 要么不成功 (即检索到虚结点 e_i)。因此有:
$$\sum_{i=1}^n p_i + \sum_{i=0}^n q_i = 1$$



平均比较次数：

- ❖ 通常，采用**平均比较次数**来衡量不同二叉查找树的查找效率。设在表示 $S = \{s_1, s_2, \dots, s_n\}$ 的二叉查找树 T 中，元素 s_i 的结点深度为 c_i ，查找概率为 p_i ；虚结点 e_i 的结点深度为 d_i ，查找概率 q_i 。
- ❖ 对于实结点 s_i ，所需比较次数为其所在二叉树的深度+1；对于虚结点 e_i ，所需比较次数为其所在二叉树的深度。
- ❖ 因此，平均比较次数 C 为：
$$C = \sum_{i=1}^n p_i(1+c_i) + \sum_{i=1}^n q_i d_i$$

最优二叉查找树的问题描述：

- ❖ 最优二叉查找树 (Optimal Binary Search Tree, OBST)：在所有表示有序序列 S 的二叉查找树 (BST) 中，具有最小平均比较次数的二叉查找树。



■ 示例 - 描述:

❖ 给定有序关键字集合 $S = \{s_1, s_2, s_3\}$ 。在一棵BST中:

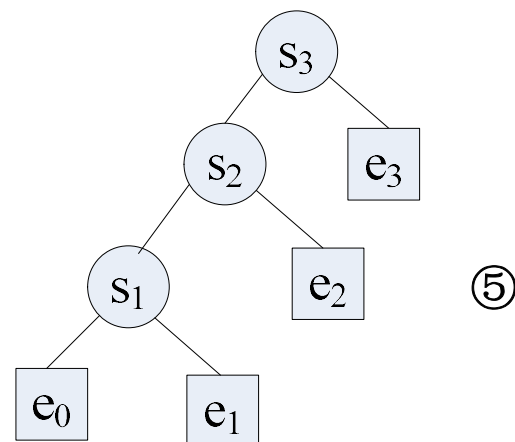
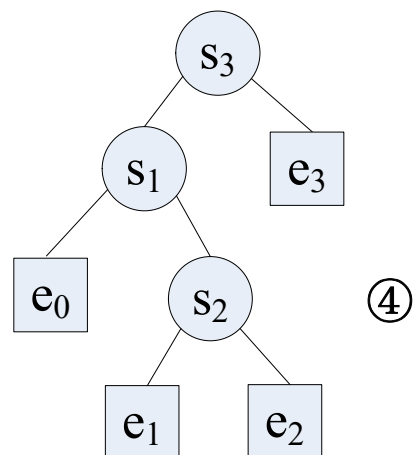
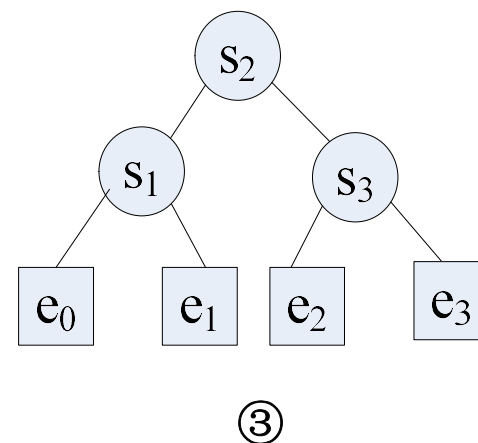
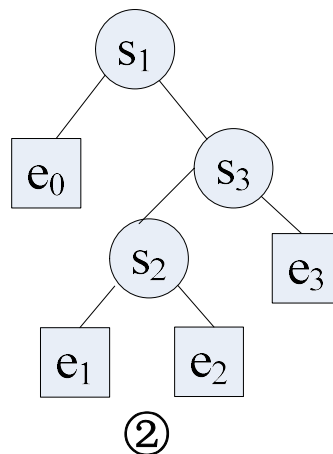
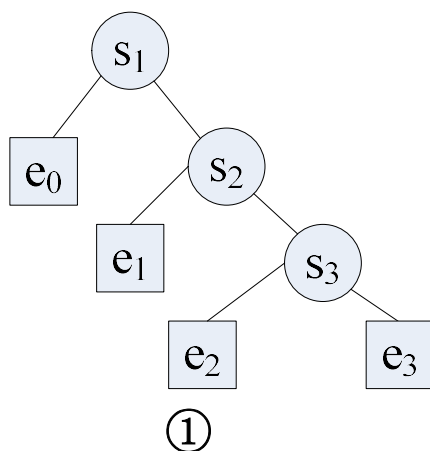
✓ 设置3个实结点 s_1, s_2, s_3 ，对应为集合 S 中的元素。设查找 $x = s_i$ 的概率为 p_i 。其中， $p_1 = 0.5$ 、 $p_2 = 0.1$ 、 $p_3 = 0.05$;

✓ 设置4个虚结点 e_0, e_1, e_2, e_3 。其中， e_0 表示所有小于 s_1 的值， e_3 表示所有大于 s_3 的值。设 e_i 对应的查找概率为 q_i 。其中， $q_0 = 0.15$ 、 $q_1 = 0.1$ 、 $q_2 = 0.05$ 、 $q_3 = 0.05$;

✓ 且有:
$$\sum_{i=1}^3 p_i + \sum_{i=0}^3 q_i = 1$$



■ 示例 - 图示：





■ 示例 - 代价：

$$C_1(n) = 1 * p_1 + 2 * p_2 + 3 * p_3 + 1 * q_0 + 2 * q_1 + 3 * (q_2 + q_3) = 1.5$$

$$C_2(n) = 1 * p_1 + 3 * p_2 + 2 * p_3 + 1 * q_0 + 3 * (q_1 + q_2) + 2 * q_3 = 1.6$$

$$C_3(n) = 1 * p_2 + 2 * (p_1 + p_3) + 2 * (q_0 + q_1 + q_2 + q_3) = 1.9$$

$$C_4(n) = 1 * p_3 + 2 * p_1 + 3 * p_2 + 1 * q_3 + 2 * q_0 + 3 * (q_1 + q_2) = 2.15$$

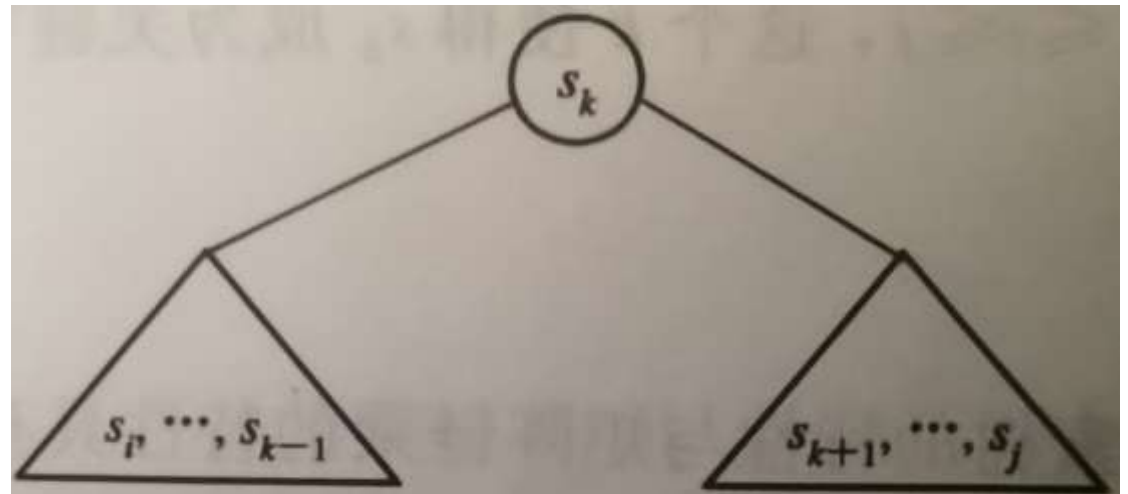
$$C_5(n) = 1 * p_3 + 2 * p_2 + 3 * p_1 + 1 * q_3 + 2 * q_2 + 3 * (q_0 + q_1) = 2.65$$

❖ → 第一棵二叉搜索树的代价最小，即为OBST。



■ OBST的最优子结构性质分析:

- ❖ 将由实结点 $\{s_1, s_2, \dots, s_n\}$ 和虚结点 $\{e_0, e_1, \dots, e_n\}$ 构成的二叉查找树记为 $T(1, n)$ 。设元素 s_k 作为该树的根结点, $1 \leq k \leq n$ 。则 $T(1, n)$ 的左子树由实结点 $\{s_1, \dots, s_{k-1}\}$ 和虚结点 $e_0, \dots, e_{k-1}$ 组成, 记为 $T(1, k-1)$; 而右子树由实结点 $\{s_{k+1}, \dots, s_n\}$ 和虚结点 $e_k, \dots, e_n$ 组成, 记为 $T(k+1, n)$ 。
- ❖ 如 $T(1, n)$ 为OBST, 其左子树 $T(1, k-1)$ 和右子树 $T(k+1, n)$ 亦为OBST。这里, 可采用反证法证明。





■ OBST的最优值递推关系:

- ❖ 已知 $T(1, n)$ 是由 $\{s_1, s_2, \dots, s_n\}$ 和 $\{e_0, e_1, \dots, e_n\}$ 所构成的最优二叉查找树。由元素 $\{s_i, \dots, s_j\}$ 和 $\{e_{i-1}, e_i, \dots, e_j\}$ 构成的最优二叉查找子树为 $T(i, j)$ 。假设选择 s_k 为根节点, 将 $T(i, j)$ 分割为左右两棵子树, 即为 $T(i, k-1)$ 和 $T(k+1, j)$ 。若使用 $r[i][j]$ 表示子树 $T(i, j)$ 的平均比较次数, 则:

$$r[i, j] = p_k + (r[i, k-1] + w(i, k-1)) + (r[k+1, j] + w(k+1, j))$$


$$r[i, j] = \begin{cases} q_{i-1} & , j = i-1 \\ \min_{i \leq k \leq j} \{r[i, k-1] + r[k+1, j] + w(i, j)\} & , j \geq i \end{cases}$$

其中, $w(i, j) = \sum_{l=i}^j p_l + \sum_{l=i-1}^j q_l$



■ 所有点对的最短路径问题相关的几个概念：

- ❖ $G=(V, E)$ 为一个有向图。其中，每条边 (i, j) 都具有一个非负的长度 $l(i, j)$ ；如从顶点 i 到顶点 j 不存在边，则 $l(i, j)=\infty$ 。
- ❖ 这里，假设 $V=\{1, 2, \dots, n\}$ ， i 和 j 是 V 中两个不同的顶点。另定义 $d^k_{i,j}$ 是从 i 到 j 且不经过 $\{k+1, k+2, \dots, n\}$ 中任何顶点的最短路径的长度。则：
 - ✓ $d^0_{i,j} \leftarrow l(i, j)$;
 - ✓ $d^1_{i,j}$ 是从顶点 i 到 j 且除了经过顶点 1 以外不经过任何其他顶点的最短路径；
 - ✓ $d^2_{i,j}$ 是从 i 到 j 且除了可能经过顶点 1、顶点 2 或同时经过顶点 1 和 2 以外不经过任何其他顶点 的最短路径。



■ APP-SP问题的最优值的递归式：

- ❖ $d_{i,j}^n$ 是从 i 到 j 的最短路径的长度（亦即 i 到 j 的距离）。其递归式如下：

$$d_{i,j}^k = \begin{cases} l[i, j] & \text{若 } k = 0 \\ \min\{d_{i,j}^{k-1}, d_{i,k}^{k-1} + d_{k,j}^{k-1}\} & \text{若 } 1 \leq k \leq n \end{cases}$$

■ 所有点对的最短路径问题的描述：

- ❖ 所有点对的最短路径（Shortest Paths of All Point Pairs, APP-SP）问题：找出从每个顶点出发到其他顶点的最短距离。



■ 使用Floyd算法求解APP-SP问题：

- ❖ Floyd算法使用自底向上地求解如上递归式。采用 $n+1$ 个 $n*n$ 维矩阵 $D_0, D_1, \dots, D_n$ 来计算约束路径的长度。
- ❖ 开始时，如果 $i \neq j$ 并且 (i, j) 是 G 中的边，则 $D_0[i, i] \leftarrow 0$, $D_0[i, j] \leftarrow l[i, j]$ ；否则， $D_0[i, j] \leftarrow \infty$ ；
- ❖ 然后做 n 次迭代，使在第 k 次迭代后， $D_k[i, j]$ 含有从 i 到 j 且不经过编号大于 k 的任何顶点的最短路径的长度。
- ❖ 这样，在第 k 次迭代中，可使用如下公式计算 $D_k[i, j]$ ：

$$D_k[i, j] = \min \{ D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j] \}$$



■ 使用Floyd算法求解APP-SP问题的示例：

例 7.5 考虑图 7.4 所示的有向图

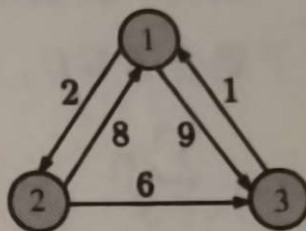


图 7.4 所有点对最短路径问题的实例

矩阵 D_0, D_1, D_2 和 D_3 是

$$D_0 = \begin{bmatrix} 0 & 2 & 9 \\ 8 & 0 & 6 \\ 1 & \infty & 6 \end{bmatrix} \quad D_1 = \begin{bmatrix} 0 & 2 & 9 \\ 8 & 0 & 6 \\ 1 & 3 & 0 \end{bmatrix}$$

$$D_2 = \begin{bmatrix} 0 & 2 & 8 \\ 8 & 0 & 6 \\ 1 & 3 & 0 \end{bmatrix} \quad D_3 = \begin{bmatrix} 0 & 2 & 8 \\ 7 & 0 & 6 \\ 1 & 3 & 0 \end{bmatrix}$$

其中， D_3 存有所有点对的最短路径。

另外，一个重要的发现：第 k ($k > 1$) 次迭代中第 k 行第 k 列相比于第 $k-1$ 次迭代都是不变的。因此，可仅用 D 矩阵的一个副本来进行计算。

Floyd 算法就是使用一个 $n \times n$ 矩阵来执行该计算的。



■ Floyd算法的伪代码:

FLOYD-APP-SP(I, D)

//输入: $I[1..n, 1..n]$ 对应于有向图 $G = (\{1, 2, \dots, n\}, E)$ 中的边的长度 $I[i, j]$ 。

//输出: $D[1..n, 1..n]$ 使得 $D[i, j]$ ($d_{i,j}^k$) 为 i 到 j 的距离。

$D \leftarrow I$ //将输入矩阵 I 复制到 D

FOR $k \leftarrow 1$ TO n

FOR $i \leftarrow 1$ TO n

FOR $j \leftarrow 1$ TO n

$D[i, j] \leftarrow \min\{D[i, j], D[i, k] + D[k, j]\}$

RETURN D

❖ 时间复杂性:

✓ $\Theta(n^3)$

❖ 空间复杂性:

✓ $\Theta(1)$

